

From Vibe Coding to Agentic Engineering

Two-Phase Technical Coursebook & Reference Manual: Vibe Coding Foundations, Context Physics & Harness Engineering (agy & claude)

Author: Alejandro Kaspar — AI Forward Deployed Engineer · Licensed under CC BY 4.0 & Apache 2.0

Two-Phase Curriculum Architecture (8 Modules, 8 Self-Diagnosing Python Labs & 10 Visual Blueprints)

- **Section 1 — Phase 1 (Modules 1 & 2): Natural Language to Code, 2022–2026 Timeline & Agentic Loop [Labs 01–02]:** Joshi & Fowler’s *What Is Code?*, Wittgenstein’s *Two Theories of Language* (*Picture Theory* vs. *Language-as-Action*, arXiv:1606.02447, arXiv:2302.01570), Andrew Ng’s **4-Pillar AI Engineering Skills Map** & Software Engineering Fundamentals, the **2022–2026 Architectural Timeline** + Anthropic’s 5-Stage Evolution (METR 4-month doubling to 12–16 hrs), aisuite 3-Layer Stack, openworker Earned Autonomy, and Stanford CS146S Wireshark harness trace inspection (Swiss-Army-Knife tools vs. 150-tool bloat).
- **Section 2 — Phase 1 (Modules 3 & 4): The RePPIT Workflow, Context Window Physics & The 40% “Dumb Zone” [Labs 03–04]:** Task Complexity Spectrum, /grill-me & Plan Mode, **Orthogonal Proposals** + **Post-Proposal Context Reset**, Implementation Plan Artifacts with Out of Scope guards, Phase-Based Model Routing, translation-agent Reflection, NextAction = LLM(ContextWindow), **Smart Zone (0–40%) vs. Dumb Zone (>40% Context Rot)**, autoresearch output-redirection backpressure, and **Frequent Intentional Compaction** (RPI) + the Human Leverage Pyramid.
- **Section 3 — Phase 2 (Module 5): Harness Engineering Layer 1 — Context Primitives, Rules & Auto-Memory [Lab 05]:** Böckeler & Morris’s Agent = Model + Harness **2×2 Cybernetic Control Matrix**, the **150-Instruction Compliance Cliff**, OpenAI & CS146S’s **Minimalist 60-Line Map Rule** (CLAUDE.md <-> AGENTS.md symlink + @docs/*.md lazy imports), path-scoped rules, and Cross-Session Memory (MEMORY.md vs. KNOWLEDGE.md).
- **Section 4 — Phase 2 (Module 6): Harness Engineering Layer 2 — Skills, context-hub & Meta-MCP “Code Mode” [Lab 06]:** agentskills.io 3-Level Progressive Disclosure, andrewyng/context-hub (@aisuite/chub) versioned API docs & chub annotate loop, CS146S **4 Rules of Ergonomic MCP Tools & Meta-MCP search** + `execute` (>90% token savings), SkillsBench (arXiv:2602.12670: +16.6 pp curated vs. −8.1 to −11.5 pp self-generated), and eval-viewer + Signal Detection Theory (d' , c).
- **Section 5 — Phase 2 (Modules 7 & 8): Subagent Firewalls, LLM Councils, Governance, Ratchets & Program Design [Labs 07–08]:** Subagent context isolation (50k → 500 tokens), 7-Phase feature-dev, CS146S Multi-Agent PR Triage (Must Fix / Recommended / Consider, confidence ≥ 80), Karpathy’s 3-Stage Anonymized `llm-council`, git worktree isolation, openworker **4-Tier Governed Harness** (PreToolUse Exit Code 2, Earned Autonomy, Provenance, NVIDIA OpenShell), OpenAI Remediation Linters ([VIOLATION] -> [WHY] -> [HOW TO FIX]), Karpathy’s autoresearch (program.md, Simplicity Criterion, Git Ratchet), Horthy’s **Why Software Factories Fail** (4-Phase Front-Loaded Program Design), and Anthropic’s 52× Kernel RSI Loop.

Contents

1	Phase 1 (Modules 1 & 2): Natural Language to Code, The 2022–2026 Timeline & The Agentic Loop (agy & claude 101)	3
1.1	Why Natural Language Correlates with Code, Wittgenstein’s Two Theories & Software Engineering Fundamentals	3
1.2	The Verified 2022–2026 Architectural Timeline & Anthropic’s 5-Stage Evolution	4
1.3	The 4-Pillar AI Engineering Skills Map, agy & claude 101, and CS146S Harness Wire Traces	4

2 Phase 1 (Modules 3 & 4): The RePPIT Daily Workflow, Context Window Physics & The 40% “Dumb Zone”	8
2.1 Directing the Daily Workflow: Research -> Propose -> Plan -> Implement -> Verify (RePPIT)	8
2.2 Context Window Physics, The 40% “Dumb Zone” & Frequent Intentional Compaction (RPI)	9
3 Phase 2 (Module 5): Harness Engineering Layer 1 — The Three Context Primitives (CLAUDE.md / GEMINI.md), Rules & Auto-Memory	12
3.1 The Harness Equation (Agent = Model + Harness) & The 2x2 Cybernetic Control Matrix	12
3.2 The Three Context Primitives, The 150-Instruction Cliff & The Minimalist 60-Line Map Rule	13
4 Phase 2 (Module 6): Harness Engineering Layer 2 — Progressive-Disclosure Skills, context-hub Grounding & Meta-MCP “Code Mode”	16
4.1 Progressive-Disclosure Skills (agentskills.io) & Curated Self-Annotating Grounding (context-hub)	16
4.2 Ergonomic MCP Tool Design & Meta-MCP “Code Mode” (> 90% Token Reduction, Stanford CS146S Lecture 4)	16
4.3 Empirical Skill Science (SkillsBench arXiv:2602.12670), eval-viewer & Trigger Calibration	17
5 Phase 2 (Modules 7 & 8): Subagents as Context Firewalls, LLM Councils, 4-Tier Governance, Autoresearch Ratchets & Why Software Factories Fail	20
5.1 Module 7: Subagents as Context Firewalls, CS146S Multi-Agent PR Triage, llm-council & Agentic VCS (git worktree vs. Jujutsu jj)	20
5.2 Module 8: 4-Tier Governance, Remediation Linters, Autoresearch Ratchets & Why Software Factories Fail	21

1 Phase 1 (Modules 1 & 2): Natural Language to Code, The 2022–2026 Timeline & The Agentic Loop (agy & claude 101)

1.1 Why Natural Language Correlates with Code, Wittgenstein’s Two Theories & Software Engineering Fundamentals

Code as Machine Instructions and an Externalized Model of Understanding (*What Is Code?*, martinfowler.com). In “*What Is Code?*” (martinfowler.com/articles/what-is-code.html, May 2026), Unmesh Joshi and Martin Fowler explain why software engineering rigor becomes *more* critical as large language models make generating syntax virtually free. Source code is not merely a sequence of instructions for a CPU; it is an externalized **model of understanding** that captures domain vocabulary, bounded contexts, and system invariants. When engineers passively accept “vibe-coded” diffs without maintaining that conceptual model, repositories rapidly accumulate **Cognitive Debt**: synonym drift across modules (inventing `client_id`, `cust_acct`, and `user_id` for the same entity), IEEE-754 floating-point currency bugs instead of integer minor units (MoneyCents), and plausible-looking abstractions that silently violate domain invariants.

Wittgenstein’s Two Theories of Language: Formal Picture Theory vs. Language-as-Action. Why can conversational natural language steer a deterministic compiler and trigger software actions at all? Ludwig Wittgenstein’s two philosophies of language illuminate the fundamental bridge between human intent and executable code (Molino & Tagliabue, [arXiv:2302.01570](https://arxiv.org/abs/2302.01570); Winograd & Flores, *Understanding Computers and Cognition*, 1986):

1. **Early Wittgenstein** (*Tractatus Logico-Philosophicus*, 1921) — **The Picture Theory of Language**: Language is modeled as a strict, formal, 1-to-1 logical picture of reality. This corresponds directly to **programming languages, static type checkers, compilers, and deterministic test suites**, where every expression is either formally valid or rejected.
2. **Late Wittgenstein** (*Philosophical Investigations*, 1953) — **Meaning as Use & Language-as-Action**: Wittgenstein later recognized that natural language is not a static logical calculus, but an active **collaborative toolbox used between coworkers to coordinate actions** (like a builder and assistant calling out “*Block!*”, “*Pillar!*”, “*Slab!*”). Words acquire rigorous meaning from their rules of use inside a shared practice (SciTePress 139777; MaKTO [arXiv:2501.14225](https://arxiv.org/abs/2501.14225)).
3. **Empirical Validation in Human–Computer Coding Interaction** (Wang, Liang & Manning, *ACL 2016*, [arXiv:1606.02447](https://arxiv.org/abs/1606.02447)): In “*Learning Language Games through Interaction*”, Stanford researchers modeled Wittgenstein’s builder–assistant paradigm in SHRDLURN across 100 human participants who could only use natural language to instruct an AI agent that executed the underlying actions. Two human habits mathematically predicted task completion: **(a) Avoiding Synonyms** (enforcing a consistent domain vocabulary) and **(b) Compositionality** (packaging low-level steps into reusable, named operational abstractions—the exact foundation of agent **Skills**). When shared context is incomplete, an epistemically honest agent pauses to ask clarifying questions rather than guessing defaults (Graziano, LGDL, 2025).

Andrew Ng’s Core Insight: Software Engineering Fundamentals Are Mandatory for Vibe Coding. In DeepLearning.AI’s *The AI Engineering Skills Map* (deeplearning.ai/resources/ai-engineering-skills#map, Aug–Sep 2026), Andrew Ng identifies the primary trap of naive Vibe Coding: a novice who prompts an agent without **Software Engineering Fundamentals** (Pillar 02) cannot spot when the agent makes disastrous architectural tradeoffs across **latency, availability, data consistency, security, reliability, maintainability, simplicity, and cost**. Because the LLM optimizes for immediate surface plausibility, the human engineer must act as the **Conductor** (*Orchestration over Automation*), supplying explicit non-functional constraints (p99 latency bounds, idempotency keys, ACID isolation, and

bounded retry budgets) and governing the agent through a progressive **Ladder of Earned Autonomy** (andrewyng/openworker).

Andrej Karpathy’s Vibe Coding (Feb 2025), MenuGen & Software 3.0 (The LLM OS). When Andrej Karpathy coined “**Vibe Coding**” on February 2, 2025—describing building weekend prototypes like *MenuGen* or *karpathy/llm-council* (“99% vibe coded as a fun Saturday hack”) by “fully giving in to the vibes, embracing exponentials, and forgetting that the code even exists”—he simultaneously framed the rise of **Software 3.0**: natural language prompts as a programmable layer atop the **LLM Operating System** (where the *Context Window* = *RAM*, *Tool/MCP Calls* = *System Calls*, and *Disk Artifacts* = *Persistent Storage*). Moving from a weekend hack to production software requires transitioning from unsteered prompting to disciplined **Harness Engineering** across Phase 1 and Phase 2 of this manual.

1.2 The Verified 2022–2026 Architectural Timeline & Anthropic’s 5-Stage Evolution

Six Architectural Eras of AI-Assisted Engineering Paired with Anthropic’s 5-Stage Evolution. In “*When AI Builds Itself*” (anthropic.com/institute/recursive-self-improvement, 2026), Jack Clark and Marina Favaro document a 5-stage industry progression (*Manual Coding [2021–2023]* → *Chatbots [2023–2025]* → *Coding Agents [2025–2026]* → *Autonomous Long-Horizon Agents [Today]* → *Closing the Recursive Loop*). Figure 1 and Table 1 pair Anthropic’s empirical benchmarks—including the **METR 50%-reliable autonomous task horizon doubling every ≈ 4 months**—from 4 minutes in 2024 to **12–16 hours in 2026**, **>80% AI-authored production code**, **8 $\times$ daily LOC output per engineer**, and open-ended session success rising from **26% to 76%–91%**—with our verified 6-Era Architectural & Human-Interaction Timeline:

Figure 1.1 Blueprint: 2022–2026 Timeline & METR 4-Month Doubling Curve

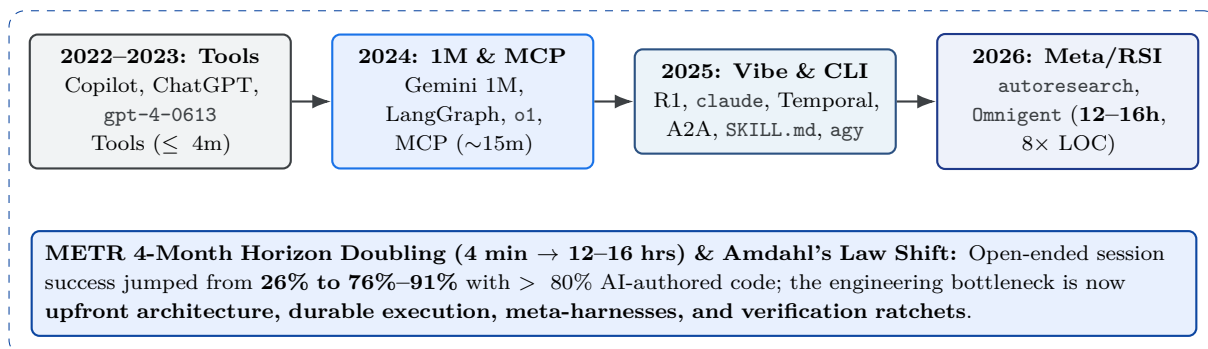


Figure 1: The verified 6-Era Architectural Timeline (2022–2026) paired with the METR 4-month autonomous task-horizon doubling trajectory (4 min $\rightarrow$ 12–16 hrs) and Anthropic’s 5-Stage Evolution.

1.3 The 4-Pillar AI Engineering Skills Map, agy & claude 101, and CS146S Harness Wire Traces

DeepLearning.AI’s 4-Pillar AI Engineering Skills Map & The 3-Layer aisuite Agent Stack. Synthesizing over 10,000 job postings, engineering leader interviews, and practitioner surveys, Andrew Ng and DeepLearning.AI organize modern AI engineering into **4 Pillars and 20 Skills** (deeplearning.ai/resources/ai-engineering-skills#map): (1) **Building & Deploying AI Applications** (LLM foundations, grounding, agent harnesses, evaluation-driven development), (2) **Software Engineering Fundamentals** (full-stack systems, data management, system architecture, shift-left security, production operations), (3) **Using Coding Agents across the 3-Stage Coding Agent Workflow** (01 Planning $\rightarrow$ 02 Execution

Era & Stage	Model, API & Protocol Breakthroughs	Human Interaction Surface	Empirical Capability Horizon
Late 2022 (Stage 1 → 2)	GitHub Copilot GA (Jun 2022), ReAct (arXiv:2210.03629, Oct 2022), ChatGPT (Nov 30, 2022).	Ghost-text Tab completion & copy-pasting code snippets from browser chat.	Single-function autocomplete (≤ 1 min horizon).
2023 (Stage 2 Chatbots)	Cursor launch (Mar 2023), OpenAI Function Calling (gpt-4-0613, Jun 13), SWE-bench (Oct), Parallel tools API (Nov 6).	In-editor @Codebase RAG & inline Cmd+K single-file diff generation.	Schema-validated JSON <code>tool_calls</code> ($\approx 2\text{--}4$ min horizon).
2024 (Long Context, Checkpointers & MCP)	LangGraph checkpointers (Jan), Gemini 1.5 Pro 1M–2M context (Feb), SWE-agent ACI (Apr), <code>.cursorsrules</code> (Jun), OpenAI o1 RL (Sep 12), Anthropic MCP (Nov 25).	Multi-file IDE Composer, stateful graph checkpointers, & static repo rule files (<code>.cursorsrules</code>).	Test-time RL compute + universal MCP “USB-C” tool bus (≈ 15 min horizon).
Early–Mid 2025 (Stage 3 Agents & Durable Workflows)	DeepSeek-R1 RLVR (Jan 20), Karpathy coins Vibe Coding (Feb 2), Claude Code + <code>CLAUDE.md</code> (Feb 24), Temporal + OpenAI Agents SDK / <code>mcp-agent</code> (Mar), Google A2A (Apr 9), Claude 4 (May 22), Gemini CLI (<code>GEMINI.md</code>) + <code>AGENTS.md</code> (Jun 25).	Terminal-native agent loops (<code>claude</code> , <code>gemini</code>) + durable workflow replay (<code>Temporal</code>); MCP (tools) + A2A (agents).	Multi-step bash/test/repair & crash-recoverable loops ($\approx 45\text{--}90$ min horizon; 26% open-ended pass rate).
Late 2025 (Skills & Antigravity)	Claude Code Subagents , Hooks & Plugins (Summer–Oct 9), Agent Skills (<code>SKILL.md</code>) (Oct 16), Google Antigravity & Gemini 3 (Nov 18), <code>agentskills.io</code> Standard (Dec 18).	Agent-first Mission Control (<code>agy</code>), Plan/Walkthrough Artifacts, Browser Subagent, 3-level <code>SKILL.md</code> .	Context-isolated subagents & progressive disclosure ($\approx 2\text{--}4$ hr horizon).
2026 – NOW (Stage 4–5 Harnesses, Meta-Harnesses & RSI)	Harness Engineering (Feb 2026: OpenAI, Fowler/Böckeler/Morris, SkillsBench), Karpathy’s autoresearch (<code>program.md</code>) (Mar), Durable Agent Harnesses (<code>Temporal</code> , <code>Restate</code> , <code>DBOS</code> , <code>Inngest</code>), Databricks Omnigent Meta-Harness (Jun 13), Antigravity 2.0 (<code>agy</code>), Ng’s Skills Map , <code>OpenWorker</code> & <code>chub</code> , CS146S RePPIT & Meta-MCP , Anthropic RSI (Aug–Oct).	Designing 4-tier governed harnesses, cross-harness meta-harnesses (<code>Omnigent</code>), durable event-replay workflows, remediation linters, & keep-or-revert ratchets.	12–16 hr METR horizon , $> 80\%$ AI code, $8\times$ LOC/day, 76%–91% session success, 52$\times$ kernel RSI .

Table 1: The complete 2022–2026 Architectural & Human-Interaction Timeline paired with Anthropic’s 5-Stage Evolution.

-> 03 Deployment & Monitoring) (directing workflows, enabling autonomy, reviewing work with browser screenshots/evals, customizing CLAUDE.md/GEMINI.md/skills/MCP/hooks, and understanding coding agent foundations), and (4) **Shaping the Build** (rapid Idea → MVP → Production small-batch loops and high-agency ownership). In andrewyng/aisuite and andrewyng/openworker, this maps onto the ****3-Layer Agent Stack**** illustrated in Figure 2 below (**Layer 1** Chat Completions API → **Layer 2** Agents API / Toolkits / MCP → **Layer 3** Governed Agent Harness).

Stanford CS146S Wireshark Harness Trace Inspection & The “Swiss Army Knife” Tool Principle. In Stanford University’s *CS146S: The Modern Software Developer* (themodernsoftware.dev, taught by Mihail Eric, Fall 2026 Lecture 2), students inspect a live production coding agent through an HTTP/JSONL proxy wire trace and contrast it against a ****200-line minimal coding agent**** (a simple while loop with 4 tools: read, write, edit, bash). Wire-trace inspection reveals why naive MCP configurations fail: loading 110–150 granular CRUD tools (such as one tool per GitHub REST endpoint) injects ****10,000+ lines of JSONL schema definitions**** for a 600-line conversation, starving the context window and confusing tool selection. Production harnesses therefore enforce the *****“Swiss Army Knife” Tool Principle****: pair a lean system prompt (≈ 50–80 lines) with a compact set of general-purpose, composable primitives (Bash, Read, Edit, Grep, Glob, LSP, SQL) that leverage the model’s post-trained CLI and Git fluency.

Figure 1.2 Blueprint: 4-Pillar Skills Map, 3-Layer Stack & CS146S Wire Trace

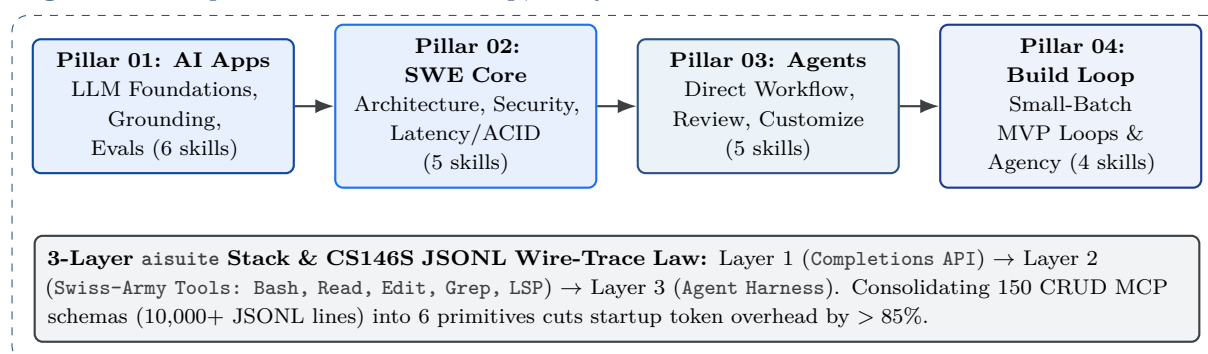


Figure 2: Andrew Ng’s 4-Pillar *AI Engineering Skills Map*, the aisuite 3-Layer Agent Stack, and Stanford CS146S’s production HTTP/JSONL wire-trace anatomy contrasting Swiss-Army-Knife primitives against 150-tool schema bloat.

Google Antigravity (agy) 101 & Anthropic Claude Code (claude) 101 Surface Mastery. Both official curricula—Google’s *Getting Started with Google Antigravity & Hands-on with Antigravity CLI* (codelabs.developers.google.com) and Anthropic Academy’s *Claude Code 101* (academy.claude.com/courses/claude-code-101)—implement the ****Core Agentic Loop** (Gather Context -> Take Action -> Verify Results)** and the ****3-Stage In-Editor Interaction Cycle**** (Stage 1: Implicit IDE state vs. Explicit @file pinning → Stage 2: Plan vs. Execute, queued messages, real-time tool monitoring → Stage 3: Interactive diff acceptance):

Dual-Branch Quickstart (agy & claude 101) & The Ladder of Earned Autonomy (Labs 01–02)

```

# 1. Clone & run the <ls stdlib self-diagnosis across all 8 progressive engineering labs
$ ./self_diagnose_all.sh
# [ALL PASSED] 8/8 Hands-On Vibe-Coding-Course Labs Verified!

# 2. Google Antigravity (agy branch): Mission Control, IDE (Cmd+L, Tab-to-Jump) & CLI
$ agy models                                # List frontier & fast models
$ agy -p "Audit latency & retry bounds" --model gemini-3.1-pro
# ~/.gemini/antigravity-cli/settings.json permission modes:
#   "request-review" -> "proceed-in-sandbox" -> "always-proceed" -> "strict"
  
```

```
# Commands: /plan, /artifact, /grill-me, /browser, /goal, /schedule, /config, !

# 3. Anthropic Claude Code (claude branch): Terminal, IDE & Desktop/Web
$ claude -p "Trace tool schema token overhead" --model claude-opus-4-7
# Shift+Tab cycles permission modes (.claude/settings.json):
#   Manual ("default") -> Auto-Accept ("acceptEdits") -> Plan ("plan") -> Auto ("auto")
# Commands: @file, ! (Bash), /context, /compact, /clear, /rewind (Esc+Esc), /btw
```

System Design Invariant & Tie-Breaker

Labs 01 & 02 Walkthrough: Architecture-Guided Steering, Earned Autonomy & Wire-Trace Profiling

- **Lab 01 — Vibe Coding vs. Architecture-Guided Steering & The Ladder of Earned Autonomy:** Contrasts naive vibe-coded prompts against software-engineering-steered prompts (enforcing explicit p99 latency budgets, integer MoneyCents domain types, idempotency keys, and clarification checks on missing FX rules) and validates the 4 permission tiers across ~/.gemini/antigravity-cli/settings.json (agy) and .claude/settings.json (claude).
- **Lab 02 — The Core Agentic Loop & Wireshark-Style Harness Trace Inspector:** Parses a multi-turn HTTP/JSONL agent wire trace across system, CLAUDE.md/GEMINI.md, tool definitions, and messages; quantifies why consolidating 150 granular CRUD tools into 6 Swiss-Army-Knife primitives cuts startup schema token overhead by > 85%; and verifies model routing (-model) and runaway-loop circuit breakers.

2 Phase 1 (Modules 3 & 4): The RePPIT Daily Workflow, Context Window Physics & The 40% “Dumb Zone”

2.1 Directing the Daily Workflow: Research -> Propose -> Plan -> Implement -> Verify (RePPIT)

The Task Complexity vs. Workflow Rigor Spectrum (Skills Map 3.1 & Stanford CS146S Lecture 3). A core skill in Pillar 03 of DeepLearning.AI’s *AI Engineering Skills Map* (3.1 Directing the workflow) and Stanford CS146S (themodernsoftware.dev, Fall 2026 Lecture 3) is matching workflow ceremony to blast radius. Applying a zero-shot prompt to a multi-module feature fails just as reliably as running a 5-stage architecture review for a CSS typo. Engineers calibrate rigor across four tiers:

- **Tier 1 — Copy / Typo / Single-Line Tweak:** Direct vanilla prompt with immediate diff review.
- **Tier 2 — Small Bounded Multi-File Change:** Single-pass Plan Mode (/plan in agy or Shift+Tab Plan Mode in claude) before editing.
- **Tier 3 — Medium Feature or Stateful Endpoint:** The full 5-step RePPIT Workflow (Research -> Propose -> Plan -> Implement -> Test/Verify) with post-proposal context reset.
- **Tier 4 — Large Cross-Cutting Refactor or Migration:** Multi-session stacked vertical slices with isolated read-only exploration and review subagents.

Defeating “Silent Assumption Syndrome” via Socratic Inversion of Control (/grill-me & Plan Mode). When given an underspecified prompt, an unsteered coding agent suffers from **Silent Assumption Syndrome**: it silently invents database schemas, authentication policies, and pagination defaults to complete the turn. Both Google Antigravity (via /grill-me, documented in the official *Build and Deploy to Google Cloud with Antigravity* codelab) and Claude Code (via Plan Mode questioning) enforce **Inversion of Control (IoC)**: the engineer instructs the agent to act as a Socratic interviewer that surfaces edge cases, failure modes, and missing requirements *before* drafting a single line of architecture or code.

The 5-Step RePPIT Workflow, Orthogonal Proposals & Post-Proposal Context Reset. Formalized in Stanford CS146S Fall 2026 Lecture 3 and integrated with Anthropic’s Explore -> Plan -> Code -> Commit workflow (*Claude Code 101*) and Andrew Ng’s **Reflection** pattern (andrewyng/translation-agent), the daily RePPIT loop illustrated in Figure 3 executes five disciplined steps:

1. **Research (research_codebase):** Read-only discovery (@file, Grep, Glob) that strictly documents *what exists today* with exact file:line citations—explicitly forbidden from proposing solutions—to build both the agent’s grounded context and the **human engineer’s mental model**.
2. **Propose (2 Orthogonal Proposals + Post-Proposal Context Reset):** Instruct the agent to generate **two genuinely orthogonal architectural proposals** (e.g., Proposal A: synchronous in-memory token bucket vs. Proposal B: distributed SQLite/Redis sliding window) with explicit trade-off matrices. **Crucial CS146S Context Hygiene Rule:** *The moment the human selects Proposal B, immediately persist Proposal B to disk and reset/clear the context window (/clear) so the rejected Proposal A tokens do not remain in attention and contaminate implementation.*
3. **Plan (Implementation Plan Artifact + Out of Scope Guardrails):** Generate a file-and-line-targeted implementation plan (Antigravity’s Implementation Plan Artifact with Google Docs-style inline comments and **Proceed** gate, or Claude Code Plan Mode via Ctrl+G) that includes an explicit **Out of Scope / Do NOT Touch** section to block agent scope creep.
4. **Implement with Phase-Based Model Routing:** Use a **frontier reasoning model** (gemini-3.1-pro or claude-opus-4-7) for *Research, Propose, and Plan*, then route *Implement* to a **fast execution model** (gemini-3.1-flash or claude-sonnet-4-6) once all architectural ambiguity is eliminated.

5. **Verify, Reflect & Commit (translation-agent Reflection + /browser Run Receipts):** Run deterministic tests, apply Andrew Ng’s 3-step ****Reflection loop** (Generate → Reflect → Refine, andrewng/translation-agent)** where the agent audits its own diff against domain invariants, verify live UI/API behavior via Antigravity’s /browser Subagent + Walkthrough Run Receipt Artifact (or Playwright MCP screenshots), and use time-travel recovery (Undo changes up to this point in agy or /rewind [Esc+Esc] in claude) if a branch diverges.

Figure 2.1 Blueprint: The 5-Step RePPIT Workflow & Context Reset

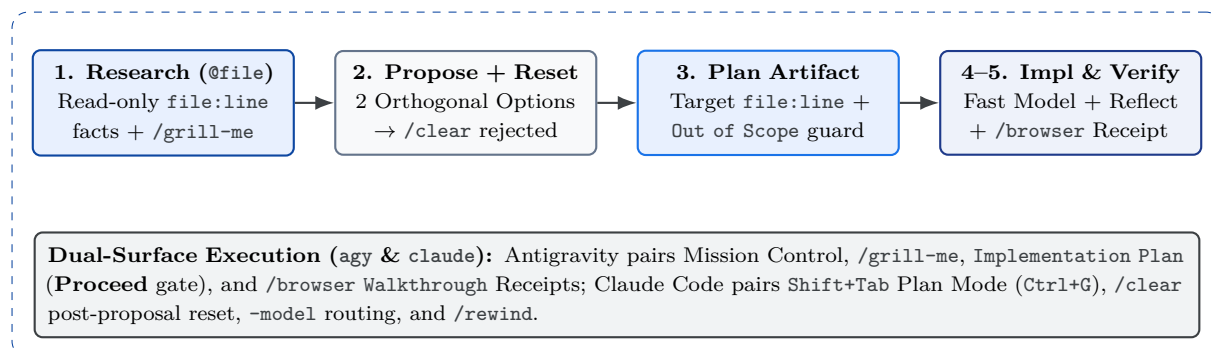


Figure 3: Five-step Phase 1 daily workflow (Research → Propose → [Context Reset] → Plan → Implement → Verify) across Google Antigravity (agy) and Claude Code (claude).

2.2 Context Window Physics, The 40% “Dumb Zone” & Frequent Intentional Compaction (RPI)

Stateless LLM Physics & The 40% “Dumb Zone” Threshold (No Vibes Allowed). In *“No Vibes Allowed: Solving Hard Problems in Complex Codebases”* (youtube.com/watch?v=rmvDxxNubIg), Dex Horthy (HumanLayer) formalizes the first law of Context Engineering: an LLM is a completely stateless next-token predictor where $\text{NextAction} = \text{LLM}(\text{ContextWindow})$. Because transformer self-attention computes pairwise interactions across every token in the window, *not all 1M tokens are created equal*. As plotted in Figure 4, agent reasoning exhibits a sharp phase transition at ****40% context utilization****:

- **The “Smart Zone” (0%–40% Context Utilization):** System instructions, CLAUDE.md/GEMINI.md invariants, and pinned file:line snippets receive high attention weight; tool calls are surgical and accurate.
- **The “Dumb Zone” (> 40% Context Utilization / Context Rot):** Once raw grep dumps, 2,000-line stack traces, and failed debugging turns push utilization past 40%, attention diffuses (*Context Rot*). The agent forgets root constraints, hallucinates function signatures, and enters repetitive error loops.

Repository Packaging (karpathy/rendergit) & Output Backpressure (karpathy/autoresearch). Andrej Karpathy’s open-source utilities demonstrate two mechanical defenses against Context Rot:

1. **Dual Human vs. LLM Repository Views (karpathy/rendergit):** rendergit flattens a Git repository into a **Human View** (syntax-highlighted HTML with a navigable tree sidebar) alongside a compact **LLM View** (structured CXML-tagged text stripped of binary/lockfile noise for high-signal context ingestion).
2. **Context Backpressure via Output Redirection (karpathy/autoresearch program.md):** Never let verbose build or training output stream into the agent’s stdout tool response. In `program.md`, Karpathy mandates `uv run train.py > run.log 2>&1` (*“redirect everything—do NOT use tee or let output flood your context”*), followed by extracting only the 2 metric lines via `grep "^val_bpb:\|^peak_vram_mb:" run.log` (or `tail -n 50 run.log` on crash).

Frequent Intentional Compaction (FIC), The RPI Loop & The Human Leverage Pyramid. Rather than waiting for automatic compaction at 95% capacity (when the agent has already spent 20 turns inside the Dumb Zone), engineers practice **Frequent Intentional Compaction (FIC)** via the **RPI Loop (Research -> Plan -> Implement)**: each phase writes a verified, line-referenced artifact to disk (research.md → plan.md) and immediately runs /clear (or /compact [focus] /compress) so the next phase starts fresh at < 15% utilization. For tangential questions during coding, Claude Code’s /btw runs a sandboxed side-query whose tokens are automatically pruned from the conversation history. By Horthy’s **Human Leverage Pyramid** (Figure 4), 1 bad line in research.md corrupts 10 lines of plan.md and generates **1,000 lines of broken code**—making human review of research.md and plan.md the highest-leverage activity in software engineering.

Figure 2.2 Blueprint: Smart Zone vs. Dumb Zone (>40%) & RPI Compaction

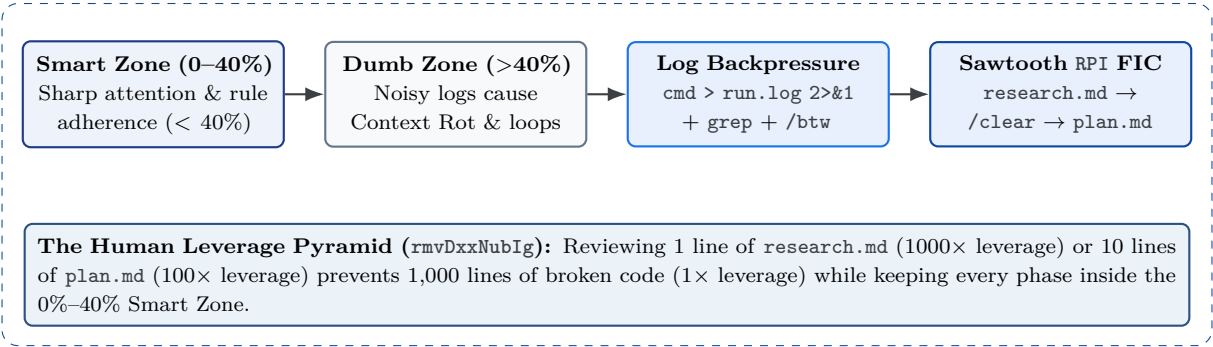


Figure 4: Context Window Physics (rmvDxxNubIg): keeping utilization inside the 0%–40% Smart Zone via output-redirection backpressure, Sawtooth RPI Intentional Compaction, and the Human Leverage Pyramid.

Workflow / Physics Step	Google Antigravity (agy Branch)	Anthropic Claude Code (claude Branch)	Context & Engineering Invariant
Requirement Interview	/grill-me Socratic interviewer before planning.	Plan Mode (Shift+Tab) clarifying questions prompt.	Defeats Silent Assumption Syndrome before architecture.
Orthogonal Proposals	Generate Proposal A vs. B in Mission Control; /clear after choice.	Generate 2 orthogonal designs; save winner & run /clear.	Eliminates rejected-proposal token pollution in attention.
Plan & Scope Guard	/plan → Implementation Plan Artifact + Proceed button.	Plan Mode (Ctrl+G) → plan.md with Out of Scope block.	Reviewing 1 line of plan saves 100 lines of broken code.
Context Audit & FIC	/stats (utilization), /compress, /clear, > run.log 2>&1.	/context (token breakdown), /compact [focus], /clear, /btw.	Never code past the 40% Dumb Zone threshold.
Verification & Undo	/browser Subagent + Walkthrough Receipt; Undo changes.	Playwright MCP + translation-agent reflection; /rewind.	Behavioral + functional proof before git commit.

Table 2: Phase 1 daily workflow (RePPIT) and context-window hygiene controls across agy and claude.

System Design Invariant & Tie-Breaker

Labs 03 & 04 Walkthrough: End-to-End RePPIT Workflow & 40% Dumb-Zone RPI Compaction Engine

- **Lab 03 — End-to-End Research -> Propose -> Plan -> Implement -> Verify (RePPIT)**

Workflow: Executes requirement clarification (/grill-me / Plan Mode), generates 2 orthogonal architectural proposals, enforces a **post-proposal context reset**, builds a line-targeted `implementation_plan.md` with explicit Out of Scope guardrails, routes models by phase, runs Generate -> Reflect -> Refine self-critique, and emits a verified `walkthrough.md` Run Receipt.

- **Lab 04 — Context Budget Profiler & Intentional Compaction (RPI) Engine:** Profiles token utilization across system prompts, file reads, and tool outputs; flags any session crossing the **40% Dumb Zone threshold**; enforces autoresearch-style output redirection (`> run.log 2>&1`); and executes **Frequent Intentional Compaction** into a validated `research.md + plan.md` handoff bundle.

3 Phase 2 (Module 5): Harness Engineering Layer 1 — The Three Context Primitives (CLAUDE.md / GEMINI.md), Rules & Auto-Memory

3.1 The Harness Equation (Agent = Model + Harness) & The 2×2 Cybernetic Control Matrix

Formalizing Harness Engineering (Agent = Model + Harness, martinowler.com). Phase 2 transitions from interactive single-session prompting to **Harness Engineering**: designing the surrounding execution environment, standing instructions, tools, and feedback sensors that turn a stochastic foundation model into a reliable engineering system. As Birgitta Böckeler and Kief Morris formulate on martinowler.com (*Exploring Gen AI*, Feb–Mar 2026), Agent = Model + Harness. While humans retain ownership of the **Why Loop** (domain value, product intent, and architectural invariants) and delegate the nested **How Loops** to agents, every component of an agent harness falls into the **2×2 Cybernetic Control Matrix** illustrated in Figure 5:

1. **Guides (Feedforward Control — Steering Before Action) × Inferential (LLM):** Root CLAUDE.md / GEMINI.md (AGENTS.md), path-scoped .claude/rules/ & .agents/rules/, and on-demand SKILL.md manuals.
2. **Guides (Feedforward Control — Steering Before Action) × Computational (CPU):** Static type annotations, Pydantic tool schemas, container filesystem sandboxes (NVIDIA OpenShell), and tool allowlists.
3. **Sensors (Feedback Control — Verification After Action) × Computational (CPU):** Deterministic unit/adversarial test suites, PreToolUse Exit-Code-2 gates, and OpenAI-style remediation linters.
4. **Sensors (Feedback Control — Verification After Action) × Inferential (LLM):** Context-isolated code-reviewer subagents, translation-agent reflection, and Karpathy’s llm-council peer review.

Figure 3.1 Blueprint: The 2×2 Cybernetic Harness Matrix (Guides vs. Sensors × CPU vs. LLM)

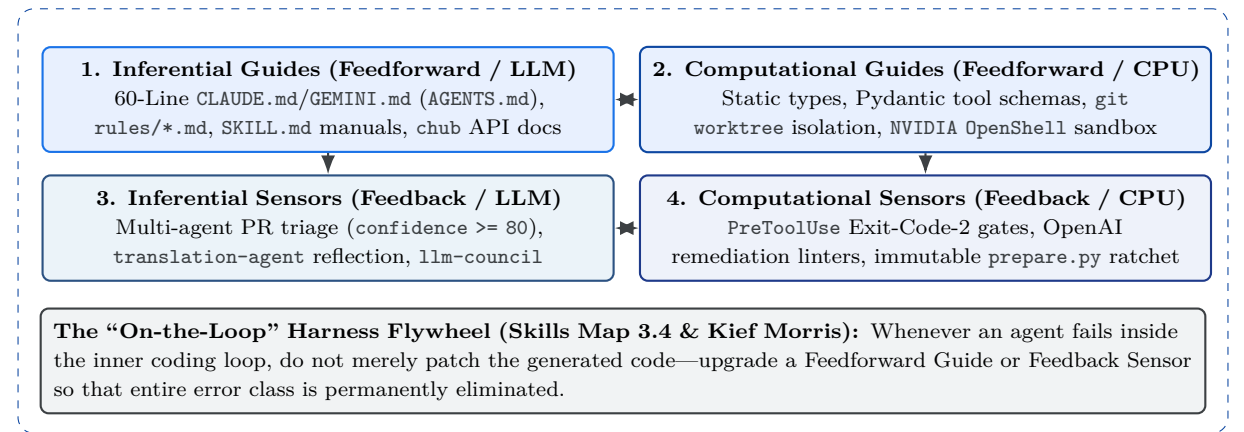


Figure 5: Birgitta Böckeler and Kief Morris’s 2×2 Cybernetic Control Matrix mapping Feedforward Guides and Feedback Sensors across deterministic CPU checks and semantic LLM reasoning.

3.2 The Three Context Primitives, The 150-Instruction Cliff & The Minimalist 60-Line Map Rule

The Three Context Primitives (Prompts, Rules, Skills) & The < 20k Standing Context Economy Rule. To prevent startup instructions from pushing the agent toward the 40% Dumb Zone before a single file is read, Harness Engineering partitions instructions across three distinct primitives (Table 3): **(1) Prompts / Slash Commands** (per-turn ephemeral task steering), **(2) Rules** (standing workspace or glob-scoped invariants in CLAUDE.md/GEMINI.md and rules/*.md), and **(3) Skills** (progressive-disclosure procedural manuals in SKILL.md). By the **Standing Context Economy Rule**, total always-on startup context (system prompt + root context file + active Level 1 skill metadata + auto-memory index) must remain strictly under ****< 20k tokens**** (< 5% of a 200k–1M context window).

The 150-Instruction Compliance Cliff & The Minimalist 60-Line Map Rule (OpenAI & Stanford CS146S). A pervasive anti-pattern in early agent adoption is treating CLAUDE.md or GEMINI.md as a 600-line dumping ground for every style preference, database schema, and historical bug fix. Empirical instruction-following benchmarks demonstrate the **150-Instruction Compliance Cliff**: LLM adherence to standing instructions remains > 92% up to ≈ 100 –150 discrete rules, then degrades sharply as competing rules dilute attention and overconstrain codebase exploration (Stanford CS146S Fall 2026 Lecture 2). Both OpenAI (*Harness Engineering: Leveraging Codex in an Agent-First World*, 2026) and CS146S enforce the **Minimalist 60-Line Map Rule** illustrated in Figure 6:

- **Keep Root CLAUDE.md / GEMINI.md to ≤ 60 Lines:** Include only (a) exact build, test, lint, and server commands, (b) a concise directory/module architecture map, and (c) 3–5 non-negotiable domain invariants.
- **Symlink CLAUDE.md $\leftrightarrow$ AGENTS.md (and GEMINI.md):** Maintain a single source of truth across Google Antigravity (agy), Claude Code (claude), and multi-tool repositories.
- **Delegate Deep Documentation via @docs/*.md Lazy Imports & Path-Scoped Rules:** Reference deep architecture or testing docs via @docs/architecture.md (0 startup tokens until pulled on demand) and scope file-specific rules using .claude/rules/*.md (paths: globs) and .agents/rules/*.md (trigger: glob | model_decision).

Figure 3.2 Blueprint: 60-Line Root Map, Lazy @docs/ & Auto-Memory

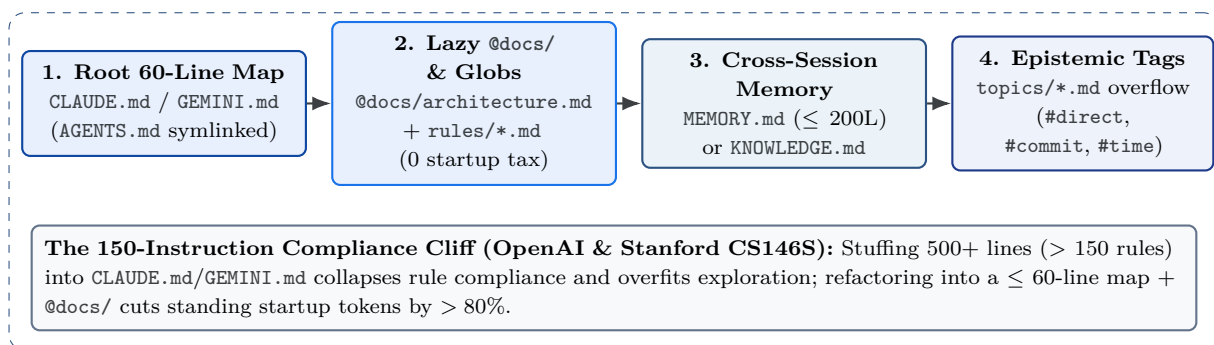


Figure 6: Defeating the 150-Instruction Compliance Cliff by pairing a ≤ 60 -line root CLAUDE.md/GEMINI.md (AGENTS.md) map and lazy @docs/*.md imports with cross-session Auto-Memory (MEMORY.md / KNOWLEDGE.md).

Cross-Session Auto-Memory (MEMORY.md vs. Antigravity KNOWLEDGE.md) & Epistemic Provenance.

While CLAUDE.md and GEMINI.md are human-authored and checked into Git, **Auto-Memory** captures agent-discovered learnings, debugging patterns, and user corrections across sessions:

- **Claude Code Auto-Memory (claude branch):** Stored at ~/.claude/projects/<project>/memory/

MEMORY.md. Because <project> is derived from the **git repository root**, all git worktree checkouts and subdirectories share one unified memory store. Only the first **200 lines / 25 KB** of MEMORY.md are auto-loaded at startup; detailed notes live in topics/*.md and are managed via /memory.

- **Google Antigravity Knowledge Items (agy branch):** Stored at ~/.gemini/antigravity/knowledge/<project>/KNOWLEDGE.md (scope: workspace) with topic files in topics/*.md, managed via /memory show|add|refresh. Every recorded fact carries strict **epistemic provenance tags**: #direct (reserved *exclusively* for explicit user instructions, never agent inferences), #commit:<sha>, #time:<YYYY-MM-DD>, and #session:<id>.

Context Primitive	Google Antigravity (agy Branch)	Anthropic Claude Code (claude Branch)	Token Budget & Scope Rule
Root Map File	GEMINI.md & symlinked AGENTS.md.	CLAUDE.md & symlinked AGENTS.md (/init).	≤ 60 lines; exact build/test/lint commands + module map.
Lazy Doc Imports	@docs/architecture.md on-demand references.	@docs/architecture.md & @docs/testing.md.	0 startup tokens until pulled for relevant task.
Path-Scoped Rules	.agents/rules/*.md (trigger: glob model_decision).	.claude/rules/*.md (paths: ["src/api/**"]).	Loaded only when matching files enter context.
Auto-Memory Store	~/.gemini/antigravity/knowledge/<proj>/KNOWLEDGE.md + #direct.	~/.claude/projects/<repo>/memory/MEMORY.md (worktree-shared).	≤ 200 lines / 25 KB index; overflow in topics/*.md.

Table 3: 1-to-1 architectural correspondence across the Three Context Primitives and Auto-Memory in agy and claude.

Copy-Pasteable Template: Minimalist 60-Line Root Map (CLAUDE.md / GEMINI.md) & Provenance Memory (Lab 05)

```
# Settlement Platform - Root Map (Symlinked: CLAUDE.md <-> AGENTS.md <-> GEMINI.md)
## 1. Exact Build, Test & Lint Commands
- Run all unit & adversarial tests (<1s): './self_diagnose_all.sh'
- Single lab: 'CI=true python3 -m unittest discover -s labs/lab_05_context/tests'
## 2. High-Level Architecture Map
- 'src/domain/': Integer value objects ('AccountId', 'MoneyCents'; NO floats).
- 'src/api/': Idempotent handlers ('.claude/rules/api.md', '.agents/rules/api.md').
## 3. On-Demand Deep Documentation (0 Startup Tokens Until Pulled)
- Domain invariants & state machines : @docs/architecture.md
- Deterministic test & hook rules : @docs/testing-conventions.md

# Cross-Session Auto-Memory Entry Format (MEMORY.md / KNOWLEDGE.md):
- Use integer minor units (MoneyCents) across payloads #direct #time:2026-10-09
- Batch worker uses SQLite WAL mode #commit:8f3a19c #session:b42e10a
```

System Design Invariant & Tie-Breaker

- Lab 05 Walkthrough: Context Compiler, 60-Line Root Map & Persistent Memory Architecture
- **Refactor Monolithic Prompts into a ≤ 60-Line Map:** Compiles a bloated 500-line instruction file into a concise root CLAUDE.md / GEMINI.md (AGENTS.md symlinked) map plus @docs/*.md lazy imports and path-scoped .agents/rules/ & .claude/rules/, verifying > 80% startup token reduction and < 150 total standing rules.

- **Enforce Git-Worktree Memory Sharing & #direct Provenance:** Resolves canonical git repository roots across worktrees, enforces the 200-line / 25 KB index cap by splitting overflow into topics/*.md, and blocks agent-inferred observations from carrying the #direct tag.

4 Phase 2 (Module 6): Harness Engineering Layer 2 — Progressive-Disclosure Skills, context-hub Grounding & Meta-MCP “Code Mode”

4.1 Progressive-Disclosure Skills (agentskills.io) & Curated Self-Annotating Grounding (context-hub)

The agentskills.io 3-Level Progressive Disclosure Architecture. An Agent Skill is a modular, filesystem-backed procedural manual (`.agents/skills/<name>/SKILL.md` in agy and `.claude/skills/<name>/SKILL.md` in claude) that teaches an agent how to execute a repeatable engineering workflow. To prevent a library of skills from exhausting the < 20k standing token budget at session startup, the open **Agent Skills Specification** (<https://agentskills.io/specification>) enforces the **3-Level Progressive Disclosure** hierarchy shown in Figure 7:

1. **Level 1 — Startup Metadata (≈ 100 tokens per skill):** Only the YAML frontmatter name (1..64 lowercase alphanumeric/hyphen characters matching the directory name) and description (≤ 1024 characters specifying positive Use when... triggers and negative Don't use for... boundaries) are loaded into the startup system prompt.
2. **Level 2 — Core Procedural Manual (SKILL.md ≤ 500 lines / < 5,000 tokens on trigger):** Loaded into context only when invoked explicitly via `/skillname` or matched by the router.
3. **Level 3 — On-Demand Scripts & Reference Manuals (scripts/, references/, assets/ — 0 startup tokens):** Domain sub-guides and deterministic verification scripts are read or executed on demand, keeping unused branches out of the context window.

Curated Versioned Grounding & Self-Improving Annotations (andrewyng/context-hub / @aisuite/chub). Even frontier models frequently hallucinate deprecated SDK methods or webhook parameters when third-party APIs evolve faster than pretraining cutoffs. Andrew Ng's open-source **context-hub** (github.com/andrewyng/context-hub, @aisuite/chub) and companion `get-api-docs/SKILL.md` (andrewyng/context-hub-skill) pair progressive disclosure with a ****self-improving session memory loop****:

- **Versioned, Language-Specific API Retrieval:** Rather than scraping noisy web pages, the agent executes `chub search` and `chub get openai/chat -lang py` to retrieve curated, version-pinned Markdown documentation.
- **Self-Annotating Learning Loop (chub annotate):** When an agent discovers a non-obvious API gotcha during a session, it persists a local annotation via `chub annotate stripe/api "Needs raw body for webhook verification"`. Subsequent sessions fetching `stripe/api -with-annotations` automatically receive the verified note (sandboxed as untrusted input by default) and can report doc quality upstream via `chub feedback up|down`.

4.2 Ergonomic MCP Tool Design & Meta-MCP “Code Mode” (> 90% Token Reduction, Stanford CS146S Lecture 4)

Why Naive 1-to-1 REST-to-MCP CRUD Wrappers Fail. In Stanford CS146S Fall 2026 Lecture 4 (themodernsoftware.dev), Mihail Eric analyzes why auto-generating one MCP tool per REST endpoint cripples coding agents. Traditional REST APIs expose low-level CRUD primitives designed for high-frequency CPU code; when an LLM is forced to call 4 sequential CRUD tools to answer one question, intermediate JSON blobs flood the context window and push the session past the 40% Dumb Zone.

The 4 Rules of Ergonomic Agent Tools & Meta-MCP “Code Mode” (search + execute). Production MCP servers follow four ergonomic design laws validated in CS146S Lecture 4 (Figure 7):

1. **Rule 1 — Design Around Complete User Outcomes, Not Granular CRUD:** Consolidate multi-step lookup chains into a single outcome tool (e.g., `resolve_and_verify_settlement`) or unify fragmented search endpoints (`search_meetings`, `search_transcripts`, `search_slack`) into `search_resource(resource_type=...)`.
2. **Rule 2 — Enforce Strict Typed Schemas (Pydantic / TypeScript) Over `**kwargs`:** Replace untyped dict payloads with strict schema constraints, enums, and bounded pagination defaults.
3. **Rule 3 — Return Actionable, Self-Repairing Typed Error Exceptions:** Format tool error strings with explicit recovery instructions so the agent repairs bad arguments on the very next turn.
4. **Rule 4 — Meta-MCP “Code Mode” (search + execute Meta-Tools for > 90% Token Reduction):** Instead of injecting 150 tool schemas (> 10,000 lines of JSONL) at startup, expose *only two meta-tools*: `search(query)` (which dynamically queries the tool registry on demand and returns only the 2–3 matching schemas) and `execute(code)` (which executes a generated script chaining those calls inside an isolated sandbox VM and returns only the distilled output), cutting MCP schema token overhead by **> 90%**.

Figure 4.1 Blueprint: 3-Level agentskills.io, chub & Meta-MCP Code Mode

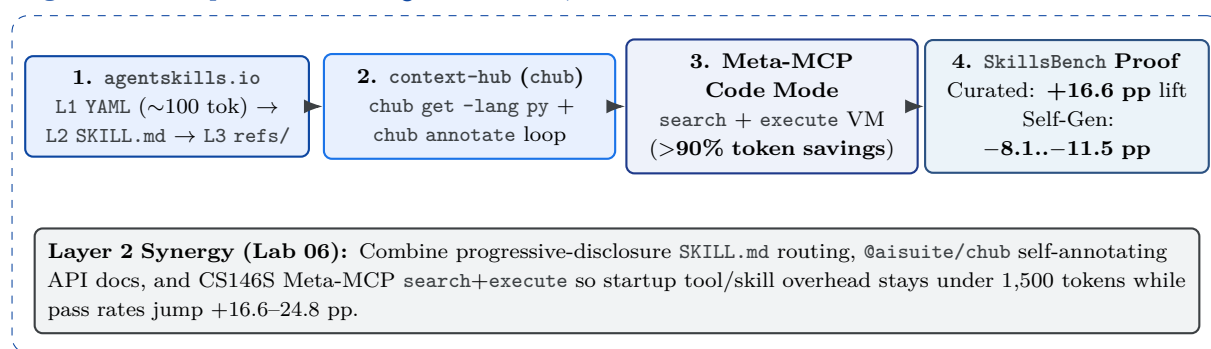


Figure 7: Layer 2 Harness Engineering combining 3-Level Progressive Disclosure (agentskills.io), context-hub (chub) self-annotating API docs, CS146S Meta-MCP “Code Mode” (> 90% token savings), and SkillsBench empirical gains.

4.3 Empirical Skill Science (SkillsBench arXiv:2602.12670), eval-viewer & Trigger Calibration

Empirical Proof from SkillsBench (arXiv:2602.12670): Curated (+16.6 pp) vs. Self-Generated (−8.1 to −11.5 pp). In *SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks* (arXiv:2602.12670v4, Li et al., Jun 2026), researchers evaluated **87 tasks across 8 domains** and **9,396 trajectories** across Claude Code, Gemini CLI, and Codex CLI (Table 4). Human-curated skills grounded in observed failure trajectories raise average pass rates from **33.9% to 50.5% (+16.6 pp)**—including **+24.8 pp on Gemini CLI + Gemini 3.1 Pro** (36.0% → 60.8%) and **+18.2 pp on Claude Code + Opus 4.7** (43.0% → 61.2%). Conversely, instructing an LLM to **self-generate its own skill before solving drops pass rates below the No-Skills baseline across every harness** (−8.1 pp on Claude Code, −11.3 pp on Codex, −11.5 pp on Gemini CLI) because zero-shot self-generated skills lock in hallucinated rules (such as multiplying unitless 3D mesh coordinates by 1000.0 in 3d-scan-calc).

Skill Overload Penalty, The 3-Stage Skill Lifecycle, eval-viewer & Signal Detection Theory (d' , c). In SkillsBench Tables 8–9, providing 2–3 focused skills achieves peak lift (**+19.0 pp**) while ≥ 4 skills

drops lift to **+10.1 pp** (−8.9 pp) and > 10 overlapping skills cause severe router confusion; similarly, Compact (**+19.0 pp**) and Standard (**+21.5 pp**) skills dramatically outperform Comprehensive dumps (**+0.7 pp**). Teams govern skills via a ****3-Stage Maturity Lifecycle**** (Figure 8): (1) *Author from observed zero-shot failure deltas*, (2) *Benchmark via paired with_skill vs. without_skill runs in Anthropic’s eval-viewer (grading.json, benchmark.json, 11/11 binomial zero-flake gate)* while calibrating trigger descriptions via Hautus (+0.5) Signal Detection Theory ($d' = \Phi^{-1}(\text{HR}_{\text{adj}}) - \Phi^{-1}(\text{FA}_{\text{adj}}) > 2.0$, criterion bias $|c| \leq 0.25$), and (3) *Prune obsolete skills when newer frontier models absorb them natively* (Skills Map 3.4).

Figure 4.2 Decision Tree: chub vs. Meta-MCP vs. SKILL.md vs. references/

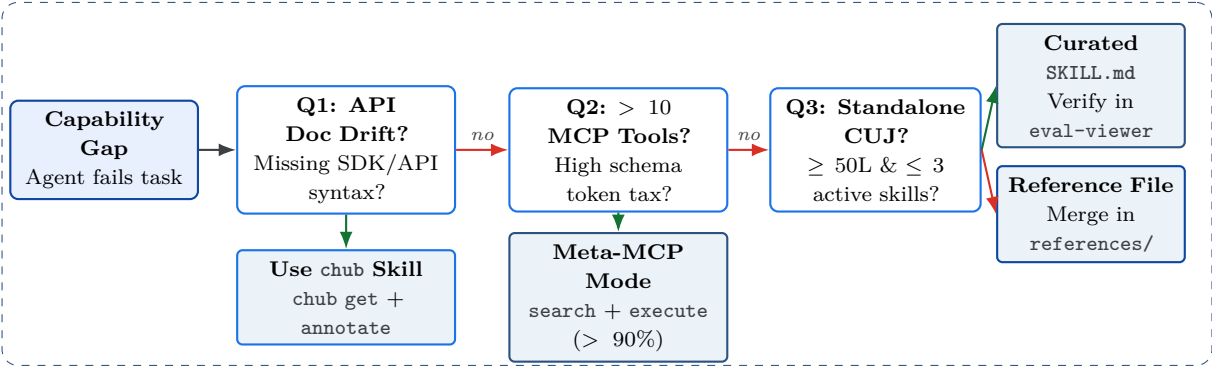


Figure 8: Left-to-Right decision tree routing capability gaps across context-hub (chub), Meta-MCP “Code Mode”, standalone SKILL.md packages, and consolidated references/*.md files.

System Design Invariant & Tie-Breaker

Lab 06 Walkthrough: Progressive-Disclosure Skills, chub Self-Annotating Docs & Meta-MCP “Code Mode”

- **Multi-File agentskills.io Validator & chub Grounding:** Validates Level 1–3 progressive disclosure (.agents/skills/ & .claude/skills/) and implements a versioned API documentation fetcher with persistent chub annotate session notes and chub feedback signals.
- **Meta-MCP “Code Mode” (search + execute) & benchmark.json:** Replaces a 120-endpoint CRUD tool catalog with dynamic search + sandboxed execute meta-tools, verifying ****> 90% schema token reduction****, SDT trigger discriminability ($d' > 2.0$), and positive net pass-rate lift in benchmark.json.

Benchmark / Mechanism	Optimal Regime	Degraded / Trap Regime	Verified Engineering Rule (Lab 06)
SkillsBench Authoring (arXiv:2602.12670 Table 6) (33.9% → 50.5%; Gemini + 24.8 pp)	Curated Skills: +16.6 pp Self-Generated: −8.1 to −11.5 pp below baseline	Never let an LLM self-generate skills zero-shot; encode only observed baseline failure deltas.	
Skill Count & Length (SkillsBench Tables 8–9)	2–3 Skills (+19.0 pp); Standard length (+21.5 pp)	≥ 4 Skills (+10.1 pp) or > 10 skills; Exhaustive (+0.7 pp)	Consolidate micro-skills into references/*.md and keep SKILL.md ≤ 500 lines.
MCP Tool Architecture (Stanford CS146S Lec 4) (> 90% schema token savings)	Meta-MCP search+execute 110–150 CRUD REST tools (> 10,000 JSONL lines)	Expose outcome tools or Meta-MCP “Code Mode” with strict Pydantic schemas.	
API Grounding & SDT (chub & eval-viewer)	chub get + chub annotate; SDT $d' > 2.0, c \leq 0.25$	Unversioned web scraping; optimizing Hit Rate alone	Persist session API quirks via chub annotate and verify 11/11 passes in benchmark.json.

Table 4: Empirical benchmarks across SkillsBench, CS146S Meta-MCP “Code Mode”, context-hub, and SDT trigger calibration.

5 Phase 2 (Modules 7 & 8): Subagents as Context Firewalls, LLM Councils, 4-Tier Governance, Autoresearch Ratchets & Why Software Factories Fail

5.1 Module 7: Subagents as Context Firewalls, CS146S Multi-Agent PR Triage, llm-council & Agentic VCS (git worktree vs. Jujutsu jj)

Why Subagents Exist: Context Firewalls (50k → 500 Tokens), Not Anthropomorphic Roleplay (rmvDxxNubIg). In Pillar 03 of DeepLearning.AI's Skills Map (3.2 Enabling agent autonomy) and Dex Horthy's *No Vibes Allowed* (youtube.com/watch?v=rmvDxxNubIg), engineers select across three interaction methods: **Conversational Prompts** (interactive co-design), **One-Off Bounded Prompts** (self-contained diffs), and **Isolated Subagents**. Subagents do not exist to roleplay corporate job titles; they exist as **Context Firewalls**. If a parent agent reads 30 files (50,000 tokens) to locate an authentication bug, its own window crosses the 40% Dumb Zone before coding starts. Delegating discovery to a child subagent burns those 50,000 tokens inside an ephemeral child window and returns a distilled **500-token summary** with exact file:line citations, preserving 99% of the parent's Smart Zone capacity.

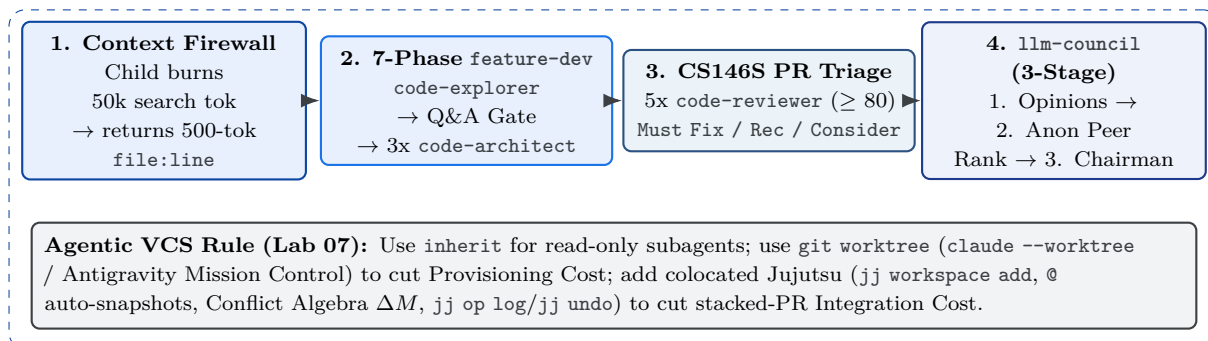
Anthropic's 7-Phase feature-dev Pipeline & CS146S Multi-Agent PR Review. Both Google AntigraVity (.agents/agents/*.md for code-explorer, code-architect, and code-reviewer, plus .agents/workflows/feature-dev.md) and Claude Code (.claude/agents/*.md + /feature-dev) orchestrate specialized subagents across a **7-Phase Pipeline**: (1) *Discovery*, (2) *Parallel Read-Only Exploration* (code-explorer, restricted to Read, Grep, Glob), (3) *Mandatory Clarifying Questions Gate* (ClarificationGateError halt), (4) *Tri-Lens Architecture* (code-architect: minimal_changes, clean_architecture, and pragmatic_balance), (5) *Implementation*, (6) *Multi-Agent PR Review* (code-reviewer), and (7) *Summary*. In Stanford CS146S Fall 2026 Lecture 3 (themodernsoftware.dev), Phase 6 spawns **4–5 parallel review subagents** (Security, Performance, Architecture, Style, and Edge Cases), suppresses low-confidence nitpicks via a strict **'confidence >= 80'** gate, and classifies findings into **'Must Fix [Red] / Recommended [Yellow] / Consider [Green]'** (Figure 9).

Andrej Karpathy's llm-council: 3-Stage Anonymized Cross-Model Peer Review & Chairman Synthesis. When evaluating high-stakes architectural designs or subtle concurrency bugs, relying on a single model family risks correlated blind spots and self-preference bias. Andrej Karpathy's open-source llm-council (github.com/karpathy/llm-council) implements a 3-stage multi-model verification protocol (Figure 9): **Stage 1 (Independent First Opinions)** dispatches the prompt in parallel across diverse frontier models; **Stage 2 (Anonymized Cross-Model Peer Review)** strips model identities (Response A, Response B, Response C) and asks each model to evaluate and rank its peers without bias; and **Stage 3 (Chairman Synthesis)** aggregates the peer rankings and reviews into a final adjudicated engineering decision.

Parallel Agent Workspace Isolation & Stacked Change Management: git worktree vs. Jujutsu (jj, wavect.io & jj-vcs/jj). When 3–10 parallel coding agents execute concurrently across four cold-start stages (*object acquisition*, *working-copy materialization*, *environment hydration*, and *agent orientation*), version control faces two distinct cost equations (Kevin Riedl, wavect.io/blog/git-worktrees-vs-jujutsu-ai-coding-agents/, 2026): $\text{Cost}_{\text{prov}} = \text{Starts} \times t_{\text{cold}} \times c_{\text{runner}}$ vs. $\text{Cost}_{\text{integ}} = \text{Changes} \times t_{\text{merge}} \times c_{\text{eng}}$. Native **'git worktree'** (AntigraVity 2.0 Mission Control and claude --worktree / isolation: worktree) paired with a shared object cache or git clone --filter=blob:none slashes $\text{Cost}_{\text{prov}}$ while preserving Git LFS, submodules, and .gitattributes. However, when parallel agents continuously rewrite stacked commits or collide on shared files, Git's stateful index and abortive rebase (DETACHED HEAD / .git/rebase-merge) freeze unattended agent loops. Colocating open-source **Jujutsu (jj, github.com/jj-vcs/jj, docs.jj-vcs.dev)** via `jj git init --colocate` and `jj workspace add ../agent-2` slashes $\text{Cost}_{\text{integ}}$ through five invariants:

1. **Working Copy Is a Commit (@) & Stable Change IDs (k-z):** Every `jj` command auto-snapshots filesystem edits into `@` without `git add/stash`, while stable **Change IDs** survive rewrites (`jj new <peer_change_id>`, `jj workspace update-stale`).
2. **First-Class Conflict Algebra ($\Delta M = \text{Tree}(M) - \text{AutoMerge}(P_1, P_2)$) — Rebases Never Fail:** Conflicts store as algebraic trees ($A + (C - B)$) inside commits so rebases never abort mid-loop, descendants auto-rebase (auto-evolve) immediately, and N-parent integration stacks (`jj new changeA changeB`) rebase deterministically without “evil merge” corruption (`jj log -r "roots(mutable() & conflicts())" --no-graph`).
3. **Transactional Operation Log & Non-Interactive Reshaping:** Every repo mutation is journaled for instant `jj op log / jj undo / jj op restore <OP_ID> rollback`, paired with non-interactive `jj split -m "..."` path, `jj squash --into <REV> -u`, `jj absorb`, and `jj converge --no-interactive`.
4. **Pragmatic Hybrid Rule (git worktree + Colocated jj):** Never interleave mutating `git` and `jj` commands; use pure `git worktree` when Git LFS/submodules/.gitattributes are required, and enforce disjoint module ownership.

Figure 5.1 Blueprint: Subagent Firewalls, PR Triage, llm-council & git worktree / Jujutsu (jj)

Figure 9: Layer 3 Multi-Agent Orchestration: subagent context firewalls (50k → 500 tokens), 7-phase feature-dev, CS146S PR review triage, Karpathy’s 3-stage anonymized llm-council, and `git worktree` vs. colocated Jujutsu (`jj`) workspace isolation.

5.2 Module 8: 4-Tier Governance, Remediation Linters, Autoresearch Ratchets & Why Software Factories Fail

Andrew Ng’s openworker 4-Tier “Governed by Design” Harness & OpenAI Remediation Linters. In github.com/andrewyng/openworker and OpenAI’s *Harness Engineering* (2026), production agent plugins (`.agents/plugins/<name>/plugin.json` on `agy` and `.claude-plugin/plugin.json` on `claude`) combine lifecycle hooks (`PreToolUse`, `PostToolUse`, `Stop`) with four defensive tiers:

1. **Tier 1 — Hard Floors (PreToolUse Exit Code 2):** Irreversible or policy-violating actions (editing locked `adversarial_tests/` or `prepare.py`, running destructive shell commands) are blocked deterministically with Exit Code 2 (“*the fixer is never the only checker*”—combining deterministic AST/semgrep scanners with model review).
2. **Tier 2 — Ladder of Earned Autonomy & Reviewer Circuit Breakers:** One-off approvals graduate into scoped allowlists, while unattended loops route actions through an independent reviewer model whose circuit breaker trips on repeated denials.
3. **Tier 3 — Provenance Audit Trail:** Every tool call records auto-approved, user-approved, or denied with timestamped reviewer rationale.
4. **Tier 4 — OS/Container Sandbox (NVIDIA OpenShell) & Remediation-Aware Linters (OpenAI):** Commands execute inside an isolated container where API keys stay outside the wall, while

PostToolUse structural linters emit agent-actionable error recipes: [VIOLATION] -> [WHY] -> [HOW TO FIX].

Andrej Karpathy's autoresearch (program.md), The Simplicity Criterion & Git Keep-or-Revert Ratchets. When a domain possesses a fast, scalar objective function—such as minimizing validation bits-per-byte (`val_bpb`) in github.com/karpathy/autoresearch (≈ 12 experiments/hr, ≈ 100 overnight) or Anthropic's ****52 $\times$ CPU training kernel speedup**** in an 8-hour autonomous loop vs. a $4\times$ human baseline (anthropic.com/institute/recursive-self-improvement)—engineers deploy a ****Bounded Autonomous Ratchet**** (Figure 10):

- **3-File Harness Separation:** `prepare.py` (*Immutable Evaluation Sensor*—read-only constants and `evaluate_bpb` protected by a Hard Floor hook), `train.py` (*Bounded Mutable Target*—the single file the agent edits), and `program.md` (*Human-Authored “Research Org Code”*).
- **The Simplicity Criterion & Git Keep-or-Revert Ratchet:** Every 5-minute experiment commits a candidate diff, redirects logs (`> run.log 2>&1`), and evaluates both metric improvement and code complexity (“A *0.001 val_bpb improvement that adds 20 lines of hacky code? Probably not worth it.* A *0.001 improvement from deleting code? Definitely keep.*”). Winning commits advance the branch; regressions or crashes trigger an immediate `git reset --hard`.

Durable Execution (Temporal, LangGraph, Restate, DBOS, Inngest) & The Meta-Harness Layer (Databricks Omnigent). As autonomous agent horizons expanded from 4 minutes in 2024 to ****12–16 hours**** in 2026, fragile in-memory Python while loops and single-harness silos hit two scaling limits: first, an LLM 429 rate-limit, MCP tool timeout, or container restart at turn 18 wipes in-memory state; second, engineers running 4–5 specialized harnesses at once (claude, agy, Codex, Pi) lack cross-harness composition, stateful governance, and multiplayer sharing. Production platforms solve both via ****Durable Execution**** and the ****Meta-Harness Layer****:

- **Durable Execution (Temporal, LangGraph, DBOS, Restate, Inngest):** In OpenAI Agents SDK + Temporal and mcp-agent, the ****deterministic Agent Loop (@workflow.defn)**** is separated from ****non-deterministic LLM and MCP calls (@activity.defn)****. Every completed activity output is appended to a persistent event history so crashed workers replay prior turns without re-billing LLM calls, while `workflow.sleep()` and external ****Signals**** pause multi-day Human-in-the-Loop (HITL) approvals with zero compute cost.
- **The Meta-Harness Layer (Databricks Omnigent, omnigent.ai, Zaharia, Uhlenhuth & Zumar, Jun 2026):** Just as Kubernetes lifted fleet management above individual servers, Omnigent sits *above* individual coding harnesses (Claude Code, Codex, Pi, OpenAI/Claude Agents SDK) behind a universal *messages/files-in, text-streams/tool-calls-out* runner across three pillars: **(1) Composition (Combine)**—authoring multi-harness YAML workflows, swapping harnesses with a 1-line change, or pairing an open-source worker model with a frontier advisor (*Harvey / Genie* pattern); **(2) Stateful Contextual Control (Control)**—tracking dynamic session state to enforce causal security and cost policies beyond static allow/deny lists (e.g., escalating `git push` to require human approval after the agent runs `npm install`, restricting writes to session-created docs, pausing every \$100 of LLM spend, and ****Egress-Proxy Credential Injection**** where raw GitHub tokens never enter the OS sandbox and are injected only inside the egress proxy on approved requests); and **(3) Multiplayer Collaboration (Share)**—sharing live agent sessions via URL across terminal, web, mobile, and Modal/Daytona cloud sandboxes.

Dex Horthy's Why Software Factories Fail (Ib5GBkD555M) & 4-Phase Front-Loaded Program Design. Why do unattended “lights-off / dark software factories” succeed on bounded kernels yet collapse into unmaintainable spaghetti after 8 weeks of building enterprise products? In “*Harness Engineering is not Enough: Why Software Factories Fail*” (youtube.com/watch?v=Ib5GBkD555M), Dex Horthy diagnoses the **RLVR Reward Horizon Gap**: Reinforcement Learning from Verifiable Rewards trains models to maximize $t = 0$ immediate test passes, whereas software architecture is evaluated at $t = 6$ months when requirements

evolve. Deterministic harnesses ****raise the floor**** (catching syntax, lint, and local test regressions) but cannot raise the architectural ceiling. Building durable systems requires keeping human engineers “on the loop” via ****4-Phase Front-Loaded Alignment**** (locking types, interfaces, and call-trees in Phase 3 before coding):

1. Product → 2. Architecture → **3. Program Design (Types & Call-Trees)** → 4. Vertical Slices (1)

Figure 5.2 Blueprint: Autoresearch Ratchet, Omnigent Meta-Harness & Program Design

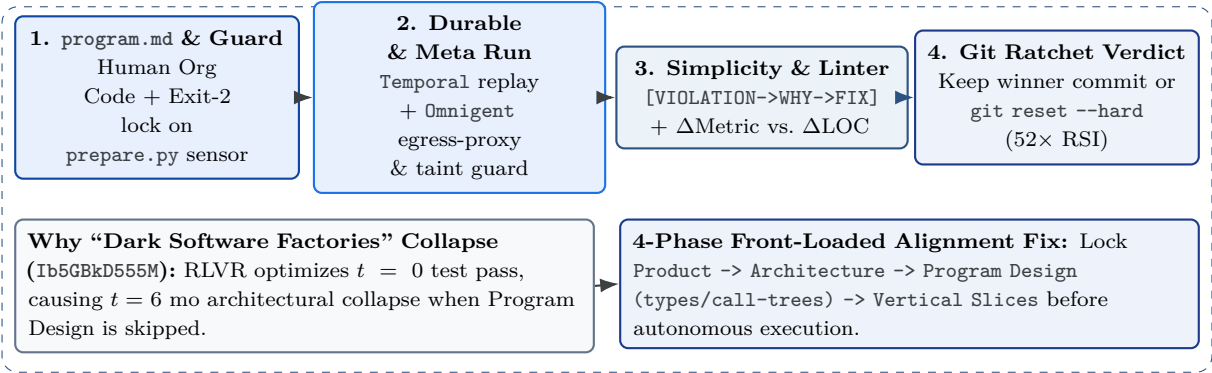


Figure 10: Layer 4 Autonomous Loops & Meta-Harnesses: Karpathy’s autoresearch (program.md) ratchet, durable execution (Temporal), Databricks Omnigent meta-harness control, OpenAI remediation linters, and Horthy’s 4-Phase Program Design.

System Design Invariant & Tie-Breaker

Labs 07 & 08 Walkthrough & Master Two-Phase Executive Takeaways

1. Lab 07 — Context-Firewall Subagents, Peer-Review Council & Agentic Workspace

Isolation: Orchestrates read-only code-explorer (50k → 500-token compression), tri-lens code-architect, CS146S Must Fix / Recommended / Consider PR triage (confidence >= 80), Karpathy’s 3-stage anonymized llm-council, and git worktree / colocated Jujutsu (jj) workspace isolation.

2. Lab 08 — 4-Tier Governance Hooks, Remediation Linters & The Autoresearch Keep-or-Revert Ratchet:

Implements an openworker PreToolUse Exit-Code-2 hard-floor gate with provenance logging, an OpenAI-style [VIOLATION] → [WHY] → [HOW TO FIX] structural linter, a 4-Phase **Program Design** type/call-tree verifier, and a Simplicity-Criterion autoresearch keep-or-revert ratchet loop.

3. Next Step in the Engineering Track:

Once you have mastered Phase 1 Vibe Coding workflows and Phase 2 Harness Engineering in this course, the next course in the track is *Spec-Driven Development* for formal multi-service specification lifecycle management.

Phase 2 Mechanism	Primary Reference	Core Invariant / Protocol	Implementation in Labs 07–08
Context-Firewall Subagents	Horthy (rmvDxxNubIg) & Anthropic feature-dev	Burn 50k search tokens in child window; return 500-token file:line digest.	code-explorer, code-architect, code-reviewer in .agents/agents/ & .claude/agents/.
Multi-Agent PR Triage & llm-council	Stanford CS146S Lec 3 & karpathy/llm-council	Filter review items at confidence ≥ 80 ; Stage 2 anonymized peer ranking.	Synthesizes Must Fix / Recommended / Consider + Chairman verdict (Lab 07).
Agentic VCS: git worktree vs. Jujutsu (jj)	Riedl (wavect.io) & github.com/jj-vcs/jj	git worktree cuts Provisioning Cost; colocated jj (@ commit, Change IDs, Conflict Algebra ΔM , jj op log/jj undo) cuts Integration Cost.	Disjoint worktree / jj workspace ownership & conflict-safe stacked diffs (Lab 07).
4-Tier Governed Harness	andrewyng/openworker & NVIDIA OpenShell	Hard Floors (PreToolUse Exit 2), Earned Autonomy, Provenance Log, Sandbox.	Blocks edits to adversarial_tests/ or prepare.py and logs provenance (Lab 08).
Remediation Linters	OpenAI <i>Harness Engineering</i> (2026)	Format linter errors as [VIOLATION] -> [WHY] -> [HOW TO FIX].	PostToolUse architectural boundary linter for autonomous self-repair (Lab 08).
Durable Execution & Meta-Harnesses	Temporal, LangGraph, DBOS, Restate, Databricks Omnigent	Event-history activity replay + Omnigent YAML composition, stateful taint/cost policies, & egress-proxy secrets.	Crash-recoverable long-horizon agent checkpoints & policy ledgers (Lab 08).
Autoresearch Ratchet & Program Design	karpathy/autoresearch & Horthy (Ib5GBkD555M)	Simplicity Criterion (Δ metric vs. Δ LOC) + 4-Phase Program Design types/call-trees.	Git Keep-or-Revert Ratchet with circuit breaker + Program Design verifier (Lab 08).

Table 5: Complete Phase 2 Harness & Meta-Harness Engineering reference matrix covering Modules 7 and 8 (Labs 07–08).